



6.10 Multidimensional Arrays (cont.)

- ▶ Figure 6.11 illustrates a two-dimensional array named **a** with three rows and four columns (i.e., a three-by-four array).
- ▶ In general, an array with m rows and n columns is called an *m -by- n array*.

	Column 0	Column 1	Column 2	Column 3
Row 0	a[0][0]	a[0][1]	a[0][2]	a[0][3]
Row 1	a[1][0]	a[1][1]	a[1][2]	a[1][3]
Row 2	a[2][0]	a[2][1]	a[2][2]	a[2][3]

Diagram illustrating the indexing of a two-dimensional array. Three arrows point to the element a[2][1] in the third row and second column:

- The outermost arrow is labeled "Array name".
- The middle arrow is labeled "Row index".
- The innermost arrow is labeled "Column index".

Fig. 6.11 | Two-dimensional array with three rows and four columns.



6.10 Multidimensional Arrays (cont.)

- ▶ Multidimensional arrays can be initialized with array initializers in declarations.
- ▶ A two-dimensional array **b** with two rows and two columns could be declared and initialized with **nested array initializers** as follows:
 - `int[][] b = { { 1, 2 }, { 3, 4 } };`
- ▶ The initial values are grouped by row in braces.
- ▶ The compiler counts the number of nested array initializers to determine the number of rows in array **b**.
- ▶ The compiler counts the initializer values in the nested array initializer for a row to determine the number of columns in that row.
- ▶ *Rows can have different lengths.*
- ▶ Multidimensional arrays are maintained as *arrays of one-dimensional arrays*.



6.13 Class Arrays (Cont.)

Java SE 8—Class Arrays Method Sort

- ▶ <https://www.geeksforgeeks.org/arrays-sort-in-java-with-examples/>



6.14 Introduction to Collections and Class `ArrayList`

- ▶ **Collections** provide efficient methods that organize, store and retrieve your data without requiring knowledge of how the data is being stored.
- ▶ The collection class `ArrayList<T>` (package `java.util`) can *dynamically* change its size to accommodate more elements.
- ▶ The `T` (by convention) is a *placeholder*—when declaring a new `ArrayList`, replace it with the type of elements that you want the `ArrayList` to hold.



6.14 Introduction to Collections and Class `ArrayList` (Cont.)

- ▶ Classes with this kind of placeholder that can be used with any type are called **generic classes**.
- ▶ Figure 6.16 shows some common methods of class `ArrayList<T>`.



Method	Description
<code>add</code>	Adds an element to the <i>end</i> of the <code>ArrayList</code> .
<code>clear</code>	Removes all the elements from the <code>ArrayList</code> .
<code>contains</code>	Returns <code>true</code> if the <code>ArrayList</code> contains the specified element; otherwise, returns <code>false</code> .
<code>get</code>	Returns the element at the specified index.
<code>indexOf</code>	Returns the index of the first occurrence of the specified element in the <code>ArrayList</code> .
<code>remove</code>	Overloaded. Removes the first occurrence of the specified value or the element at the specified index.
<code>size</code>	Returns the number of elements stored in the <code>ArrayList</code> .
<code>trimToSize</code>	Trims the capacity of the <code>ArrayList</code> to the current number of elements.

Fig. 6.16 | Some methods and properties of class `ArrayList<T>`.



```
1 // Fig. 6.17: ArrayListCollection.java
2 // Generic ArrayList<T> collection demonstration.
3 import java.util.ArrayList;
4
5 public class ArrayListCollection
6 {
7     public static void main(String[] args)
8     {
9         // create a new ArrayList of Strings with an initial capacity of 10
10        ArrayList<String> items = new ArrayList<String>();
11
12        items.add("red"); // append an item to the list
13        items.add(0, "yellow"); // insert "yellow" at index 0
14
15        // header
16        System.out.print(
17            "Display list contents with counter-controlled loop:");
18
19        // display the colors in the list
20        for (int i = 0; i < items.size(); i++)
21            System.out.printf(" %s", items.get(i));
22    }
```

Fig. 6.17 | Generic ArrayList<T> collection demonstration. (Part I of 3.)



```
23 // display colors using enhanced for in the display method
24 display(items,
25     "%nDisplay list contents with enhanced for statement:");
26
27 items.add("green"); // add "green" to the end of the list
28 items.add("yellow"); // add "yellow" to the end of the list
29 display(items, "List with two new elements:");
30
31 items.remove("yellow"); // remove the first "yellow"
32 display(items, "Remove first instance of yellow:");
33
34 items.remove(1); // remove item at index 1
35 display(items, "Remove second list element (green):");
36
37 // check if a value is in the List
38 System.out.printf("\n\"red\" is %sin the list%n",
39     items.contains("red") ? "" : "not ");
40
41 // display number of elements in the List
42 System.out.printf("Size: %s%n", items.size());
43 }
44
```

Fig. 6.17 | Generic ArrayList<T> collection demonstration. (Part 2 of 3.)



```
45 // display the ArrayList's elements on the console
46 public static void display(ArrayList<String> items, String header)
47 {
48     System.out.printf(header); // display header
49
50     // display each element in items
51     for (String item : items)
52         System.out.printf(" %s", item);
53
54     System.out.println();
55 }
56 } // end class ArrayListCollection
```

Display list contents with counter-controlled loop: yellow red
Display list contents with enhanced for statement: yellow red
List with two new elements: yellow red green yellow
Remove first instance of yellow: red green yellow
Remove second list element (green): red yellow
"red" is in the list
Size: 2

Fig. 6.17 | Generic ArrayList<T> collection demonstration. (Part 3 of 3.)



6.14 Introduction to Collections and Class ArrayList (Cont.)

Java SE 7—Diamond (<>) Notation for Creating an Object of a Generic Class

- ▶ Consider line 10 of Fig. 6.17:
 - `ArrayList<String> items = new ArrayList<String>();`
- ▶ `ArrayList<String>` appears in the variable declaration and in the class instance creation expression. Java SE 7 introduced the **diamond (<>) notation** to simplify statements like this.
- ▶ Using `<>` in a class instance creation expression for an object of a generic class tells the compiler to determine what belongs in the angle brackets.



6.14 Introduction to Collections and Class `ArrayList` (Cont.)

- ▶ In Java SE 7 and higher, the preceding statement can be written as:
 - `ArrayList<String> items = new ArrayList<>();`
- ▶ When the compiler encounters the diamond (`<>`) in the class instance creation expression, it uses the declaration of variable `items` to determine the `ArrayList`'s element type (`String`)—this is known as *inferring the element type*.